Fundação Escola de Comércio Álvares Penteado - FECAP
CIÊNCIA DA COMPUTAÇÃO - 6º SEMESTRE

Caroliny Rossi Bittencourt – 24025959
Duda Lucena Miguel – 24025889
Gabriel Debastiani Davanço – 24025899
Ricardo Liyudi Tetsuya – 24026553

ENTREGA 2 - SISTEMAS EMBARCADOS E ROBÓTICA

RELATÓRIO TÉCNICO DE PROVA DE CONCEITO (PoC)

Dispositivo Wearable para Reconhecimento de Movimentos no Tênis via IMU e IA

São Paulo – SP
2026

1 INTRODUÇÃO E DESCRIÇÃO DO DISPOSITIVO

O projeto consiste no desenvolvimento e na validação da Prova de Conceito (*Proof of Concept* – PoC) de um dispositivo eletrônico vestível (*wearable*) voltado à telemetria esportiva e análise biomecânica aplicada ao tênis. A finalidade primária do dispositivo é a aquisição contínua de variáveis cinemáticas para a posterior classificação supervisionada de golpes (e.g., *forehand*, *backhand*, saque) através de algoritmos de inteligência artificial.

Para a mensuração inercial, o sistema emprega a Unidade de Medição Inercial (*Inertial Measurement Unit* – IMU) MPU-6050, que opera com 6 Graus de Liberdade (6-DoF), fornecendo leituras tridimensionais de aceleração linear (a_x, a_y, a_z) e velocidade angular ($\omega_x, \omega_y, \omega_z$). Os dados são processados e integrados a uma cadeia computacional que engloba rotulagem de séries temporais, treinamento de modelos e inferência de padrões de movimento.

2 ARQUITETURA E FLUXO DE FUNCIONAMENTO

O ciclo de vida operacional do dispositivo organiza-se em duas etapas interdependentes:

2.1 Etapa 1: Aquisição de Dados e Modelagem de Inteligência Artificial (Offline)

A primeira etapa corresponde ao processo de extração e tratamento dos dados para treinamento do classificador estatístico/preditivo:

1. O microcontrolador lê os registradores do sensor MPU-6050 via barramento I^2C e transmite os vetores brutos pela interface Serial (*Universal Asynchronous Receiver-Transmitter* – UART/USB) em taxa amostral periódica;

2. Um script em linguagem Python captura o fluxo de dados no computador hospedeiro via biblioteca de porta serial, efetuando o desacoplamento das variáveis nos seis eixos cinemáticos;

3. Os dados são consolidados juntamente a uma coluna de índice temporal (*timestamp* / *frame*), gerando conjuntos de dados estruturados em formato `.csv` devidamente rotulados por classe de golpe para alimentação do pipeline de treinamento de modelos de aprendizado de máquina.

2.2 Etapa 2: Operação no Ecossistema Integrado e Inferência (Online)

A segunda etapa estabelece o funcionamento autônomo do dispositivo integrado ao ecossistema de aplicação do usuário:

1. O microcontrolador coleta as leituras inerciais e realiza o empacotamento em formato hexadecimal para redução de *overhead* de transmissão;

2. Os pacotes são transmitidos sem fio por meio do protocolo *Bluetooth Low Energy* (BLE) para o cliente operacional (aplicativo móvel ou cliente *web* via *Web Bluetooth API*);

3. A aplicação receptora executa a decodificação dos bytes hexadecimais, o pré-processamento de filtragem de sinal e submete os tensores de entrada ao modelo de inferência hospedado na API *backend*, retornando em tempo real a classificação do movimento executado pelo atleta.

3 MONTAGEM E PROVA DE CONCEITO DO PROTÓTIPO

A montagem física implementada corresponde a um protótipo laboratorial de prova de conceito, voltado estritamente à comprovação da viabilidade funcional de aquisição e integridade da transmissão de dados no ecossistema proposto:

- **Plataforma de Processamento:** Utiliza-se a placa Arduino UNO R4 WiFi, responsável pela gestão dos protocolos de comunicação, temporização e leitura dos periféricos;
- **Sensoriamento:** O sensor MPU-6050 é acoplado diretamente aos barramentos de comunicação I^2C (pinos SDA e SCL) e referências de tensão;
- **Alimentação Autônoma:** Para viabilizar a mobilidade sem cabos conectados à rede elétrica, o circuito é alimentado por uma associação de duas baterias de polímero de lítio (LiPo) de 3,7 V em série (totalizando 7,4 V nominais), intermediadas por módulo de gerenciamento de carga;
- **Aspectos Construtivos:** Nesta iteração de prova de conceito, o protótipo não conta com carcaça ou encapsulamento selado, operando com cabeamento direto e suporte em protoboard para viabilizar medições elétricas imediatas, inspeção de hardware e validação dos barramentos lógicos.

4 TECNOLOGIAS E FERRAMENTAS EMPREGADAS

A Tabela 1 sintetiza a especificação e o papel desempenhado por cada tecnologia de hardware e software dentro da arquitetura do projeto.

Tabela 1 – Especificação das Tecnologias e Ferramentas do Projeto

Camada / Domínio	**Componente / Tecnologia**	**Função no Sistema**
Hardware Embarcado	Arduino UNO R4 WiFi	Microcontrolador mestre, execução do firmware, processamento I^2C e transmissão BLE.
	MPU-6050	Sensor inercial de 6 graus de liberdade (aceleração linear e velocidade angular X, Y, Z).
	2x Baterias LiPo 3,7 V (Série)	Fonte de alimentação autônoma de 7,4 V nominais para operação desvinculada da rede.
	Módulo de Carregamento	Circuito integrado de proteção e recarga segura das células de lítio.
Edge / Firmware	Arduino IDE / C++	Desenvolvimento do código embarcado, leitura por interrupções e empacotamento de dados.
Frontend	React Native (TypeScript)	Aplicação cliente para pareamento Bluetooth, recepção do fluxo e interface do usuário.
Backend e Dados	FastAPI (Python)	Servidor REST assíncrono para ingestão de séries temporais e orquestração do modelo de IA.
	SQLite / TinyDB	Persistência local e estruturação dos registros de sessões, métricas e histórico de treinos.

5 DEMONSTRAÇÃO PRÁTICA DO PROTÓTIPO

Para demonstrar o funcionamento e a validação em bancada da Prova de Conceito (PoC) desenvolvida — evidenciando a alimentação autônoma por baterias LiPo, a sinalização visual via matriz de LEDs 12x8 nativa do Arduino UNO R4 WiFi e o display OLED, bem como a comunicação em tempo real —, disponibilizou-se o registro audiovisual do protótipo em operação:

- **Vídeo de Demonstração (YouTube):** https://youtube.com/shorts/tBKtHX7DJaE

6 CÓDIGO-FONTE DO SISTEMA EMBARCADO

O Código 1 apresenta a implementação completa do firmware embarcado desenvolvido em C++ para a placa Arduino UNO R4 WiFi. A lógica contempla a ini-

cialização do barramento I^2C, integração da matriz de matriz de LEDs 12x8 nativa, suporte ao display OLED SSD1306 e configuração dos serviços GATT para comunicação *Bluetooth Low Energy* (BLE).

Listing 1: Firmware do Wearable FLUX (Arduino UNO R4 WiFi)

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <ArduinoBLE.h>
#include "Arduino_LED_Matrix.h"

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET    -1

//
    ================================================================================

// METADADOS DO SISTEMA EMBARCADO (ENVIADOS APENAS 1x NO HANDSHAKE BLE)
//
    ================================================================================

#define DEVICE_NAME      "FLUX-Tennis-R4"
#define BOARD_MODEL      "Arduino UNO R4 WiFi"
#define FIRMWARE_VER     "v1.0.0"

// Instancia a Matriz de LEDs 12x8 nativa do Arduino UNO R4 WiFi
ArduinoLEDMatrix matrix;

// Bitmap 8x12: Icone "X" (Desconectado / Aguardando Bluetooth)
uint8_t ICON_DESCONECTADO_X[8][12] = {
  { 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0 },
  { 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0 },
  { 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0 },
  { 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0 },
  { 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0 },
  { 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0 },
  { 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0 },
  { 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0 }
};

// Bitmap 8x12: Icone "Check" (Conectado via Bluetooth ao App)
uint8_t ICON_CONECTADO_CHECK[8][12] = {
  { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1 },
  { 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1 },
  { 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0 },
  { 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0 },
```

```
  { 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0 },
  { 0, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0 },
  { 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0 },
  { 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0 }
};

// Instancia o display OLED no barramento I2C
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
bool oledDisponivel = false;

// ============================================================================

// CONFIGURACAO BLUETOOTH LOW ENERGY (BLE) - COMPATIVEL COM O FRONTEND FLUX
// ============================================================================

// Servico padrao de Bateria BLE (0x180F)
BLEService batteryService("180F");
BLEUnsignedCharCharacteristic batteryLevelChar("2A19", BLERead | BLENotify);

// Servico principal do FLUX Smart Tennis
BLEService fluxImuService("4fafc201-1fb5-459e-8fcc-c5c9c331914b");

// Caracteristica 1: Ficha Estatica do Aparelho (Modelo|Versao) -> Lida apenas 1x na conexao!
BLEStringCharacteristic fluxDeviceInfoChar(
  "beb5483e-36e1-4688-b7f5-ea07361b26a9",
  BLERead,
  64
);

// Caracteristica 2: Stream de Telemetria IMU durante o treino
BLEStringCharacteristic fluxTelemetryChar(
  "beb5483e-36e1-4688-b7f5-ea07361b26a8",
  BLERead | BLENotify | BLEWrite,
  64
);

unsigned long ultimoEnvioMs = 0;
int contadorPacotes = 0;
uint8_t nivelBateria = 96;
bool estavaConectado = false;

```

```
78 // Atualiza a Matriz de LEDs 12x8 da placa UNO R4 WiFi (Check =
       conectado, X = desconectado)
79 void atualizarMatrizLED(bool conectado) {
80   if (conectado) {
81     matrix.renderBitmap(ICON_CONECTADO_CHECK, 8, 12);
82   } else {
83     matrix.renderBitmap(ICON_DESCONECTADO_X, 8, 12);
84   }
85 }
86
87 // Atualiza o display OLED com o status atual do Bluetooth e telemetria
88 void atualizarDisplay(bool conectado, String enderecoCentral) {
89   if (!oledDisponivel) return;
90
91   display.clearDisplay();
92   display.setTextSize(1);
93   display.setTextColor(SSD1306_WHITE);
94
95   // Cabecalho
96   display.setCursor(0, 0);
97   display.print(DEVICE_NAME);
98   display.print(" ");
99   display.println(FIRMWARE_VER);
100  display.drawLine(0, 11, 127, 11, SSD1306_WHITE);
101
102  // Status da conexao Bluetooth
103  display.setCursor(0, 16);
104  if (conectado) {
105    display.println("BLE: CONECTADO! [OK]");
106    display.setCursor(0, 28);
107    display.print("App: ");
108    display.println(enderecoCentral);
109  } else {
110    display.println("BLE: DESCONECTADO [X]");
111    display.setCursor(0, 28);
112    display.println(BOARD_MODEL);
113  }
114
115  // Rodape com contador de pacotes enviados
116  display.setCursor(0, 44);
117  display.print("Pacotes BLE: ");
118  display.println(contadorPacotes);
119
120  display.setCursor(0, 55);
121  display.print("Bateria: ");
122  display.print(nivelBateria);
123  display.println("%");
```

```
124
125   display.display();
126 }
127
128 void setup() {
129   Serial.begin(115200);
130   delay(1000);
131
132   // 1. Inicializa a Matriz de LEDs 12x8 do UNO R4 WiFi (comeca exibindo
        'X')
133   matrix.begin();
134   atualizarMatrizLED(false);
135
136   // 2. Inicializa barramento I2C e Display OLED
137   Wire.begin();
138   Wire.setClock(100000); // 100 kHz standard
139   Wire.setTimeout(100);
140
141   Serial.println("\n--- INICIALIZANDO FLUX WEARABLE (UNO R4 WIFI) ---");
142
143   if (display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
144     oledDisponivel = true;
145   } else if (display.begin(SSD1306_SWITCHCAPVCC, 0x3D)) {
146     oledDisponivel = true;
147   } else {
148     Serial.println("[AVISO] Display OLED nao detectado. Usando Matriz de
      LEDs 12x8 + BLE.");
149   }
150
151   // 3. Inicializa o modulo Bluetooth Low Energy (ArduinoBLE)
152   if (!BLE.begin()) {
153     Serial.println("[ERRO] Falha ao iniciar o modulo Bluetooth Low
     Energy (BLE)!");
154     while (1) {
155       delay(1000);
156     }
157   }
158
159   // Define o nome visivel no navegador (Web Bluetooth) e no Smartphone
160   BLE.setDeviceName(DEVICE_NAME);
161   BLE.setLocalName(DEVICE_NAME);
162
163   // Configura e adiciona o servico de Bateria (0x180F)
164   batteryService.addCharacteristic(batteryLevelChar);
165   BLE.addService(batteryService);
166   batteryLevelChar.writeValue(nivelBateria);
167
```

```
  // Configura e adiciona o servico principal do FLUX
  fluxImuService.addCharacteristic(fluxDeviceInfoChar);
  fluxImuService.addCharacteristic(fluxTelemetryChar);
  BLE.addService(fluxImuService);

  // Registra o Modelo da Placa e a Versao do Software UMA UNICA VEZ no registrador GATT
  // Formato: 'Modelo da Placa|Versao do Software' (lido pelo Frontend apenas no handshake)
  String handshakeMetadata = String(BOARD_MODEL) + "|" + String(FIRMWARE_VER);
  fluxDeviceInfoChar.writeValue(handshakeMetadata);
  fluxTelemetryChar.writeValue("FLUX_READY");

  // Anuncia o servico principal para ser encontrado no scan
  BLE.setAdvertisedService(fluxImuService);

  // Inicia o advertising BLE
  BLE.advertise();

  Serial.print("[OK] BLE Ativo! Metadados embarcados registrados: ");
  Serial.println(handshakeMetadata);
  atualizarDisplay(false, "");
}

void loop() {
  // Escuta conexoes de um dispositivo Central (Navegador Web ou Smartphone)
  BLEDevice central = BLE.central();

  if (central) {
    if (!estavaConectado) {
      estavaConectado = true;
      Serial.print("[BLE] Conectado ao App! MAC Central: ");
      Serial.println(central.address());

      // Garante que o nivel de bateria inicial esta atualizado no handshake
      batteryLevelChar.writeValue(nivelBateria);

      // Desenha o CHECK na Matriz de LEDs 12x8 do Arduino UNO R4 WiFi!
      atualizarMatrizLED(true);
      atualizarDisplay(true, central.address());
    }

    // Enquanto o app estiver conectado ao Arduino
    while (central.connected()) {
```

```
      unsigned long agora = millis();

      // Envia apenas telemetria leve (sem reenviar modelo/versao para economizar banda!)
      if (agora - ultimoEnvioMs >= 500) {
        ultimoEnvioMs = agora;
        contadorPacotes++;

        String payload = "PKT:" + String(contadorPacotes);
        fluxTelemetryChar.writeValue(payload);

        atualizarDisplay(true, central.address());
      }

      BLE.poll();
    }

    // Quando o app desconectar, volta a desenhar o X na Matriz de LEDs 12x8!
    estavaConectado = false;
    Serial.print("[BLE] Desconectado de: ");
    Serial.println(central.address());
    atualizarMatrizLED(false);
    atualizarDisplay(false, "");
  } else {
    unsigned long agora = millis();
    if (agora - ultimoEnvioMs >= 1000) {
      ultimoEnvioMs = agora;
      atualizarDisplay(false, "");
    }
  }
}
```